# MovieLens Data Lakehouse Hands-on Exercise

Apache Spark | Apache Iceberg | Apache Polaris | SeaweedFS S3 | MySQL JDBC

Student assignment | Four PySpark notebooks | Bronze - Silver - Gold - MySQL

## 1. Learning objectives

Build a complete, repeatable data lakehouse pipeline. Ingest MovieLens CSV data into S3-compatible object storage, transform it into Apache Iceberg tables registered in Polaris, calculate popular movies, and publish the final results to MySQL. All processing must be idempotent.

## 2. Download the dataset

Use the following alternative GitHub source (the GroupLens website may be unavailable):

Download MovieLens Latest Small ZIP from training-sh GitHub

Open the link and select Download raw file. Extract the ZIP. Use only movies.csv and ratings.csv for this assignment; other included files are optional and out of scope.

## 3. End-to-end architecture

```
MovieLens ZIP (movies.csv, ratings.csv)
        |
SeaweedFS / S3: movielens/bronze/{movies,ratings}/
        |
Spark -> Polaris / Iceberg: movielens_silver
        |
Spark analytics -> Polaris / Iceberg: movielens_gold
        |
Spark JDBC -> MySQL: movielens.popular_movies
```

## 4. Task 1 - Create Bronze storage

- Create an S3 bucket named movielens.
- Create prefixes bronze/movies/ and bronze/ratings/.
- Upload movies.csv and ratings.csv into their respective prefixes.
- Verify both objects are readable from Spark using the configured S3A endpoint.
- Bronze is immutable raw CSV for this exercise; do not create Iceberg Bronze tables.

```
s3a://movielens/bronze/movies/movies.csv
s3a://movielens/bronze/ratings/ratings.csv
```

## 5. Task 2 - Notebook 01: Movies Bronze to Silver

Notebook: 01_bronze_to_silver_movies.ipynb

Create the Polaris namespace movielens_silver if absent, then create Iceberg table polaris.movielens_silver.movies. The prefix polaris is the assumed Spark catalog name; adapt only if the trainer

config differs.

| Column | Type |
|---|---|
| movieId | INT |
| title | STRING |
| genres | STRING |

• Read movies.csv with an explicit schema; reject null or invalid movieId and blank title.

• Remove exact duplicates; enforce one row per movieId.

• For conflicting rows with the same movieId, choose and document a deterministic resolution rule.

• Create the Iceberg table if missing; synchronize records using MERGE INTO or an equivalent idempotent full-snapshot strategy.

• Execute twice and demonstrate unchanged row counts and table contents.

## 6. Task 3 - Notebook 02: Ratings Bronze to Silver

Notebook: 02_bronze_to_silver_ratings.ipynb

Create Iceberg table polaris.movielens_silver.ratings.

| Column | Type |
|---|---|
| userId | INT |
| movieId | INT |
| rating | DOUBLE |
| timestamp | BIGINT |

• Read ratings.csv with explicit types; reject invalid or null key fields, ratings and timestamps.

• Exclude every rating less than or equal to 0.5.

• Remove exact duplicates. Use (userId, movieId, timestamp) as the logical composite key for this exercise.

• Resolve conflicting values under the same composite key deterministically.

• Use an idempotent synchronization approach; repeat the notebook and verify no new duplicates.

```
clean_ratings = ratings_df.filter(F.col("rating") > 0.5)
```

## 7. Task 4 - Notebook 03: Silver to Gold analytics

Notebook: 03_silver_to_gold_popular_movies.ipynb

Read both Silver Iceberg tables. Compute per-movie average rating and rating count. Keep only movies where average rating >= 4.0 and number of ratings >= 100. Join qualifying aggregates with movies to include title and genres.

```
SELECT movieId, COUNT(*) AS total_ratings,
      AVG(rating) AS avg_rating
FROM polaris.movielens_silver.ratings
```

```
GROUP BY movieId
HAVING AVG(rating) >= 4.0 AND COUNT(*) >= 100;
```

Create namespace movielens_gold and Iceberg table polaris.movielens_gold.popular_movies.

| Column | Type |
|---|---|
| movieId | INT |
| title | STRING |
| genres | STRING |
| total_ratings | BIGINT |
| avg_rating | DOUBLE |

One row per movieId is mandatory. The Gold result must be a complete reflection of the current Silver data. Use deterministic overwrite or a synchronization strategy that also removes movies that no longer meet the thresholds. Append-only and upsert-only approaches are insufficient for changed source snapshots.

## 8. Task 5 - Notebook 04: Gold to MySQL JDBC

Notebook: 04_gold_to_mysql.ipynb

Read polaris.movielens_gold.popular_movies and synchronize the results to MySQL table movielens.popular_movies via JDBC.

| Setting | Value |
|---|---|
| MySQL host | Trainer-provided hostname or IP |
| Port | 3306 |
| Database / table | movielens / popular_movies |
| Application user / password | team / team1234 |
| Admin user / password | root / team1234 |

• Create the database and table if absent; define movieId as the PRIMARY KEY.

• Use team for normal JDBC access; use root only for initial database setup or permissions.

• Do not commit credentials to notebooks or source control; use runtime configuration.

• A recommended design is Spark JDBC writing to a staging table followed by transactional MySQL synchronization (upsert plus delete of stale rows).

• Verify MySQL exactly matches Gold on movieId, title, genres, count and average; repeated runs must not duplicate records.

## 9. Mandatory duplicate handling and idempotency

| Layer | Logical key / expectation | Repeat-run behavior |
|---|---|---|
| Bronze | Raw source objects | No repeated appends |
| Silver movies | movieId | One row per movie |
| Silver ratings | userId, movieId, timestamp | One row per event key |
| Gold | movieId | Fully recomputed and synchronized |
| MySQL | PRIMARY KEY(movieId) | Fully synchronized with Gold |

Snapshot assumption: For this assignment, the CSV files represent complete source snapshots. If a source record is removed or becomes invalid, the corresponding Silver and downstream results must be corrected. A MERGE that only inserts or updates does not, by itself, satisfy this requirement.

## 10. Mandatory validation and change-data tests

- Confirm both Bronze CSV files exist and are Spark-readable.
- Confirm Silver movies count equals distinct movieId count.
- Confirm Silver ratings contain no rating <= 0.5 and no duplicate composite keys.
- Confirm every Gold row has avg_rating >= 4.0 and total_ratings >= 100.
- Confirm every Gold movieId joins to an existing Silver movie.
- Confirm MySQL row count and values exactly match Gold.
- Run each notebook twice with unchanged input; compare counts and contents.
- Add a valid rating; rerun and verify the affected aggregate updates.
- Insert a duplicate rating; verify it is counted once.
- Change a rating from 4.0 to 0.5; verify it disappears from Silver.
- Remove enough ratings for a movie to fall below 100; verify it disappears from Gold and MySQL.

## 11. Deliverables

```
movielens-project/
  notebooks/
    01_bronze_to_silver_movies.ipynb
    02_bronze_to_silver_ratings.ipynb
    03_silver_to_gold_popular_movies.ipynb
    04_gold_to_mysql.ipynb
  sql/mysql_setup.sql
  screenshots/
    bronze_bucket.png
    polaris_silver.png
    polaris_gold.png
    mysql_popular_movies.png
  README.md
```

README.md must explain setup, notebook order, catalog and S3 configuration (without secrets), schemas, deduplication rules, synchronization strategy, rerun tests and observed results.

## 12. Evaluation rubric

| Assessment area | Weight |
|---|---|
| S3 / Bronze setup | 10% |
| Movies Silver processing | 15% |
| Ratings Silver processing and filter | 15% |
| Gold analytics and Iceberg | 20% |
| MySQL JDBC integration | 15% |
| Duplicate handling and idempotency | 20% |
| Documentation and evidence | 5% |
| TOTAL | 100% |

Completion criterion: All four notebooks run in order; Gold satisfies the popularity criteria; MySQL matches Gold; repeat runs and changed-input tests demonstrate no duplicates or stale records.